

RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Anonymous Author(s)

Abstract

Conformance suites are most valuable where specifications are narrow and negative: malformed syntax, duplicate declarations, out-of-range literals, and dynamic errors. These cases are easy to name in a reference semantics and tedious to construct by hand. We present RuSMT, a framework that turns one executable semantics into both a conformance oracle and a marker-directed test-suite synthesizer. The author writes the semantics as a restricted Rust program over SMT-denoting types and marks obligations with stable names such as `Path::named("division_by_zero")`. The same source runs concretely as the oracle and lowers to one SMT-LIB reachability query per marker; each accepted model becomes a test input.

The main engineering problem is not expressing the semantics but finding inputs. On a TOML 1.1.0 parser with 182 markers, Z3 alone produced no witness within affordable budgets: symbolic parser inputs keep both sides of deeply nested branches alive during macro expansion. RuSMT therefore uses a two-stage pipeline. Stage 1 asks Z3 to synthesize the input directly. Stage 2, used only after timeout or unknown, asks a language model to propose a candidate from the emitted SMT-LIB query and Z3's previous responses. That candidate is not trusted; it is added as an equality to the original query, and only a sat result from Z3 becomes a witness. This converts the hard part of the TOML run from solver search to solver acceptance while preserving the original marker obligation.

In our run, Stage 1 found both IMP witnesses (2/2) and no TOML witnesses (0/182). Stage 2, using live gpt-5.4-mini calls and cached transcripts for replay, produced 131/182 TOML witnesses; 51 markers remain uncovered and are reported by name. The same run recorded 689 candidate proposals: 131 were accepted, 551 were rejected by Z3 as not reaching their claimed marker, and 7 were undecided. Running the generated TOML suite against four independent parsers produced 15 accept/reject divergences, including likely parser bugs and several float-boundary cases that require specification triage. The contribution is the pipeline and artifact: an executable semantics whose marked branches generate a reusable conformance suite, with Z3 providing logical acceptance and replay checking the SMT lift against the concrete oracle.

CCS Concepts: • Software and its engineering → Software testing and debugging; Formal software verification; • Theory of computation → Automated reasoning.

Keywords: executable semantics, conformance testing, bounded program synthesis, SMT, Z3, solver-aided programming, Rust, large language models

1 Introduction

A language ecosystem typically carries three artifacts meant to say the same thing: a prose *specification*, an *implementation* adhering to the specification, and a *test generator*. Written separately, they drift. Fuzzing covers the bulk of the input space but misses the narrow error and edge conditions a specification cares about most; a hand-written suite forces its author both to decide which conditions are interesting and to craft inputs exercising them. RuSMT keeps the author's judgment—expressed as markers in the semantics itself—and hands input generation to a backend solver, so when the specification changes the conformance suite is *regenerated*, not re-authored by hand—a living, executable specification that drives its own testing.

Why error cases. Markers target a language's *negative space*: its *syntactic* errors, which are ill-formed input the grammar rejects, and its *semantic* errors—well-formed input violating the static or dynamic semantics, such as an out-of-range value, a duplicate key, a type error, or a division by zero—the textbook front-end/analysis boundary [16]. The focus is empirically grounded, not a convenience of what the solver finds easy. Error-handling code is where defects concentrate and testing is thinnest: a majority of *catastrophic* distributed-systems outages trace to incorrect handling of errors, often in code a trivial test would have caught [30], and error paths are routinely dropped in Linux file systems [8]. It is also where implementations *diverge*—no two JSON parsers agree across malformed documents [21], and parser/validator discrepancies on ill-formed input are a documented security-vulnerability source [4, 20]—and what conformance suites assert normatively [6]. RuSMT therefore does not fuzz values broadly; it synthesizes inputs reaching the specification's own error and violation branches, then differentially tests how implementations treat them [13].

RuSMT collapses two of these three artifacts into one. The author writes the executable semantics *once*, in a restricted dialect of Rust whose standard library's types denote Z3's mathematical theories. That single source is given two interpretations: run as ordinary Rust it is the concrete conformance *oracle*; lifted to SMT-LIB it is the source of a solver query that *synthesizes* programs reaching designated points. The author marks a point with a named marker, e.g. `Path::named("division_by_zero")`; RuSMT asks for

an input driving execution there, decodes the result into runnable source, replays it against the oracle, and hands the witness to independent implementations. For TOML, the decoded result is already a TOML document. For IMP, Z3 produces an AST, so the framework renders the model back to an IMP program before replay and downstream use.

When the specification changes, the suite is *regenerated* by re-running synthesis against the same markers rather than re-authored by hand. The pipeline is intentionally ordered. Stage 1 asks Z3 to synthesize from the unconstrained query. If Z3 returns *sat*, the model becomes the witness; if it returns *unsat*, the marker is unreachable under the encoding and bound. Only when Stage 1 returns no useful verdict does Stage 2 ask a model for a concrete candidate from the emitted SMT-LIB and the solver’s previous feedback. That candidate is appended as `input = c`; if Z3 returns *sat*, the strengthening lemma says the candidate also satisfies the original marker query. Replay then checks that the concrete Rust oracle and the SMT lift agree on that input. A model can guide search, but it cannot approve a test.

Research questions. **RQ1:** Can a direct-style Rust interpreter, written in a small SMT-denoting DSL, serve simultaneously as conformance oracle and marker-directed test generator? **RQ2:** What discipline is sufficient for accepting a generated test without human inspection? **RQ3:** When Z3 alone fails on a realistic parser, how much coverage does the model-to-Z3 co-solving stage recover? **RQ4:** Does the Z3 acceptance step reject model proposals often enough to matter? **RQ5:** Run against independent implementations, what accept/reject divergences does the generated suite expose?

Contributions. This is a tool contribution: not a new solver, and not a completeness theorem. **(1) A Rust-native framework anyone can reuse.** You write an ordinary recursive interpreter over the DSL vocabulary and get two artifacts from one source: compiled by `rustc` it is the conformance *oracle*, and lowered to SMT-LIB it is a marker-directed *test generator*—no separate grammar, no hand-written SMT (§3–4). **(2) A concrete Z3-first, model-second pipeline.** Z3 attempts every marker first; only a timeout or unknown invokes the model, and the model sees emitted SMT-LIB plus Z3 feedback rather than the Rust source. **(3) Acceptance that needs no human inspection.** A witness is kept only when Z3 shows the lifted semantics reach the *named* marker on that input, and replay checks the SMT lift against the concrete Rust oracle (§5). **(4) Measured conformance suites for TOML 1.1.0 and IMP.** The run covers 131/182 TOML markers and 2/2 IMP markers, with all misses named (§8). **(5) A downstream implementation comparison.** The generated TOML suite is run against four real parsers and exposes 15 accept/reject divergences. **(6) A replayable artifact** with pinned tool versions, generated queries, cached model transcripts, and the exact suite used for the differential run.

2 Background

SMT as the reasoning backend. RuSMT uses SMT to ask one operational question: *does there exist an input such that this executable semantics reaches marker d?* The DSL therefore maps Rust expressions to a fixed vocabulary of Z3 theories [5]: booleans, integers/reals, bit-vectors, strings, sequences, sets, arrays, algebraic datatypes, and recursive functions encoded in SMT-LIB 2.6 [2]. For each marker the backend emits a formula of the form

$$\exists x. \text{entry}(x) = \text{Err}(\text{marker_id}(d)),$$

where x is the object-language program or input text. On *sat*, Z3 returns a model for x ; on *unsat*, no such x exists for that encoding and bound; on unknown or timeout, the framework learns only that this backend did not finish.

The scientific and engineering issue is that the generated formulas are outside a decidable quantifier-free fragment. A recursive interpreter becomes *define-funs-rec*; Z3 reasons about such definitions by unfolding and heuristics. Parsers additionally combine strings/sequences, arithmetic, bit-vectors, recursive datatypes, and occasional quantifiers handled by incomplete instantiation. Thus RuSMT treats Z3 as a sound decision procedure for returned verdicts, not as a complete search oracle: *sat* candidates must still replay through the Rust semantics, and unknown/timeout costs only coverage. The same split explains why proposer recovery is useful. Searching for an unconstrained input may be intractable, while validating a fully determined candidate can be easy because the backend macro-inlines the concrete sequence and Z3 constant-folds along one path.

Executable semantics and synthesis. A *semantics* fixes the meaning of a language; it is *executable* when that definition is itself a runnable artifact—a reference interpreter written *to be* the specification, not merely any implementation that happens to run programs (a production compiler runs the language without being its definition). Frameworks like K [18], PLT Redex [7], and Ott/Lem [14, 22] write that definition as rewrite/inference rules and derive a tool suite from them. RuSMT instead takes the semantics to be an *ordinary recursive function*: the same source executes concretely as the oracle and compiles symbolically as the search space. Compared with fuzzers such as Csmith [29], the target is not coverage in general but a named specification obligation. Compared with SyGuS and SemGuS [1, 9], the user does not author a separate grammar or CHC semantics; the unknown is an input to the interpreter the user already wrote.

3 The RuSMT DSL

A RuSMT specification is an ordinary Rust program, written once and read two ways: compiled by `rustc` it *runs* as the reference interpreter; lifted by the transpiler it *denotes* an SMT formula. The design principle is that these two readings must agree by construction, so every construct admitted into

the DSL is one with a mathematical meaning the solver can express. That single requirement dictates the whole subset: RuSMT is a deliberately *functional*, *total* fragment of Rust.

A functional subset, and why each restriction earns its place. The DSL keeps immutable `let` bindings, `if` with a mandatory `else`, `struct/enum` with `match`, and (mutually) recursive calls; it forbids loops, mutation, and references. These are not arbitrary—each exclusion buys a property the lift depends on. *No mutation or references* means a value never aliases or changes under another name, so a `let` is mathematical substitution and the translation is referentially transparent; this is the same discipline that lets a big-step operational semantics be read as a system of equations. *No loops* leaves recursion and (bounded) quantifiers as the sole means of iteration, matching how a solver expresses repetition—as recursive definitions and quantified formulas, not as mutable state evolving over time. The payoff is that the interpreter reads like the inference rules it formalizes: Winskel’s big-step rules for IMP (§7.1) transcribe almost line for line, and there is no gap between “the specification” and “the code that runs.”

Totality makes error cases first-class. The subset is, crucially, *total and exception-free*: no `panic!`, no `?`, no `.unwrap()`, no early return. A partial function has no total SMT reading, so any divergence from the happy path must be expressed as a returned *value*—for example a `ParseResult::Err` carrying a `Path` (below)—rather than a thrown exception. This is exactly what the negative-space thesis (§1) wants, and it comes for free: every error condition is, by construction, an explicit branch returning an error value, i.e. a place a marker can sit, on code that must exist anyway.

Separation of concerns. Two layers are kept strictly apart. The *standard library* (below) is a fixed vocabulary denoting *Z3*’s theories and nothing else; the *interpreter* encodes a *target* language’s semantics on top of that vocabulary. A target’s idiosyncrasies—two’s-complement wraparound, a parser’s error conditions, an uninitialized-variable rule—live in the interpreter as explicit guards, never inside the `stdlib`. This keeps the per-construct soundness obligation (§5) small, fixed, and language-independent: it is checked once for the `stdlib`, not re-argued per case study.

Mechanical enforcement. Two attribute macros enforce the subset at compile time, so an out-of-subset specification fails to build rather than mistranslating silently. `#[smt_type]` admits only `structs` and `enums` whose every value is `Copy` (so nothing aliases); `#[smt_fn]` rejects `const/async/unsafe` and variadic functions, method receivers, and any body beyond `let` bindings and a trailing expression.

Listing 1 shows the whole discipline in one IMP fragment—the division guard of the arithmetic evaluator. It is ordinary recursive Rust; the only synthesis-specific token is `Path::named`, sitting on the error branch a total evaluator must have anyway. The primitive `bv_div` already agrees with *Z3* on every nonzero divisor, so the interpreter guards only the diverging case (a zero divisor) and falls through to `bv_div` otherwise—the guard/if-then-else pattern of §5. From this single source RuSMT synthesizes `A := (0 / 0)` (§8); replayed through the same `eval_aexp`, it reaches the marker.

```
if *new_r.eq(I64::from(0)) {
    ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
    ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Listing 1. The division guard of IMP’s evaluator (operands already evaluated to `new_l`, `new_r`; `Err` propagation elided).

Standard library: the theory vocabulary. The DSL’s types denote *Z3* sorts: Boolean, unbounded Integer/Real (big integers/rationals), the bit-vectors `I32/I64/U32/U64` (the signed/unsigned split is a DSL-API reading of the same *Z3* sort), IEEE `F32/F64`, String (the Strings theory), `Seq<T>`, `Set<T>`, and `Array<K,V>`. As established above, these denote *Z3*’s theories, not Rust’s native semantics and not any target language’s. Two helper types are frontend-only. `Cloak<T>` is a Box-like indirection that lets an author write self-referential ADTs (e.g. `Add(Cloak<Aexp>, Cloak<Aexp>)`) despite Rust’s sized-type rule; the IR erases it entirely—every `Cloak<T>` field lowers to `T` and shield/reveal to identity—since *Z3* supports recursive datatypes natively (the one proviso is well-foundedness: a `Cloak<T>` must co-occur with a non-recursive variant, so the lowered datatype is inhabited). `Path`, the marker type, is described next.

Markers via Path. A marker flags a program point the author wants covered—characteristically an *error or specification-violation* branch (the “negative space” of §1); markers are *syntactic* or *semantic* (the taxonomy above), and the case studies span both. The value a marker carries is a `Path`, and its *identity* is what lets the marker be matched across the two readings of the specification. An id drawn from a global counter would be order-dependent, and so unmatchable against a replayed run; instead `Path::named("d")` derives its id as a fixed hash of the name *d*, and the *same* pure function serves transpiler and evaluator, so the id the query asserts and the id a re-executed witness carries are *identical by construction*. In the SMT backend a `Path` is represented by that id as a plain Integer (with a sentinel for “no marker reached”). The choice is deliberate: an integer equality remains decidable through the deep recursive unfolding of a lifted interpreter, whereas a set-membership test would force the solver to construct

and probe a symbolic collection it cannot evaluate on those paths—an encoding decision we return to in §4. Each named marker is thus one synthesis target (§6); named markers are what make per-target acceptance marker-specific (§5), and the case studies we replay use them.

Quantifiers. `forall!`, `exists!`, and `choose!` exist in a *bounded* form over a collection (`forall!(x in xs => p)`). `choose!` is Hilbert choice: concretely the first satisfying witness, in SMT a Skolem function under the standard choose axiom—sound because nothing may depend on *which* witness is chosen, only that it satisfies the predicate.

4 The compilation pipeline

The same annotated Rust source is compiled in two modes. In oracle mode, `rustc` executes the interpreter on concrete candidate inputs. In solver mode, the transpiler parses the restricted Rust subset, lowers it to a sort-checked IR by Hindley–Milner unification over SMT sorts, monomorphizes generic functions on demand, records each `Path::named` marker, and emits a per-marker query. Listing 2 shows the backend loop. Every query is inspectable SMT-LIB2, which is what makes each reported result re-runnable with a bare `z3 -smt2` on a committed file.

```

335 for each named marker d in entry:
336   Q = lower_to_smtlib(entry, marker_id(d), k)
337   r = run("z3 -smt2 -t:<budget_ms>", Q.main)
338   if r == sat:
339     c = decode_model(r.model)
340     if replay(c, d) == ReachedTarget:
341       accept(c)
342   else:
343     report_replay_rejection(d, c)
344     continue
345
346 if r == unsat:
347   report_unreachable_under_encoding_or_bound(d)
348   continue
349
350 // Stage 2: model proposes, Z3 decides.
351 for round in 1..R:
352   c = model(reads: Q.main, d, history)
353   v = run_z3(Q.main + assert(input_0 == c), budget)
354   if v == sat && replay(c, d) == ReachedTarget:
355     accept(c)
356     break
357   feedback(v, replay(c, d))

```

Listing 2. Backend loop for one entry function. Stage 1 uses the emitted query unchanged. Stage 2 runs only after timeout or unknown; it pins a proposed candidate as one extra equality and asks Z3 to decide the strengthened query.

Backend and budgets. The backend renders inspectable `.smt2` files from the IR and drives `z3` as a subprocess. Stage 1 uses the unconstrained marker query exactly as emitted. Stage 2 is reached only after Stage 1 times out or reports unknown; the model receives the SMT-LIB text, the target marker name, and the previous rejected candidates and Z3 verdicts. It runs in an empty working directory with file and shell tools disabled, so it cannot inspect the Rust semantics. The proposed input is pinned by adding one equality to the

original query; `unsat` means only that this candidate does not reach this marker, while `sat` gives a marker-specific witness. The reported TOML run used the candidate-acceptance mode with `RUSMT_ROUNDS=9`, `RUSMT_WITNESSES=1`, a 15 s Z3 budget for pinned candidates, and cached model transcripts. IMP uses native recursion (`k=0`); TOML also uses `k=0` because the free input is the document text.

Engineering for Z3. Several encodings are deliberately solver-facing. A marker is an Integer `id`, not a symbolic set membership test, because equality survives deep recursive unfolding. Recursive maps such as TOML tables are encoded as array-free inductive association lists; an `Array` whose range is a recursive datatype made even concrete-candidate validation return unknown in early TOML experiments. Recursion is offered either as native `define-funs-rec` or as fixed-depth unrolling because neither dominates across the case studies. The backend parses the leading Z3 status line before trusting process exit status, since Z3 can exit nonzero after benign model-output failures. A `sat` model is then rendered back to object-language source by re-sharing `let` terms, unmangling constructors, canonicalizing solver-chosen names, and decoding bit-vectors as two’s-complement integers.

5 Soundness of the lift

The trusted computing base. The trusted base is the RuSMT specification together with the transpiler (the lift to SMT) and Z3. We *assume* the prose-to-DSL transcription correct (trusted, not verified), and we trust the transpiler and solver *because* the lift is faithful—the per-construct contract below, *checked* rather than asserted—so trusting Z3’s verdict reduces to that one obligation. The concrete parser and pretty-printer are also trusted; a bug in any is a bug in the results. Replay is not what discharges this trust: it is a per-candidate cross-check that the symbolic and concrete readings agree, catching a faithfulness defect on any candidate it touches as a spurious rejection. Per-target acceptance additionally assumes `marker_id` is injective over the finite, author-declared set of marker names—as a 63-bit hash, a checkable side condition trivially met by the names we use.

The per-construct contract. Soundness rests on a contract for every standard-library primitive:

Definition 5.1 (Per-construct soundness contract). For every `stdlib` function f and every concrete input x ,

$$\text{rust_impl}(f, x) = \text{z3_eval}(\text{z3_formula}(f), x).$$

The `stdlib` denotes Z3’s theories; bridging it to a given target language is the interpreter author’s job.

The guard / if-then-else pattern. A primitive matching the target language is used directly; one that diverges only on specific inputs is *guarded* on those inputs, falling through to the (Z3-correct) primitive otherwise. Listing 1 has this shape:

`bv_div` agrees with Z3 everywhere, so the only special case is a zero divisor, which the guard turns into a marker. Because concrete Rust and Z3 take the *same* branch on the same input, Definition 5.1’s equality survives the conditional—and the same discipline appears inside the `stdlib` (e.g. a width-or-more shift returns 0 to match Z3’s `bvshl`).

How the contract is validated. Validation is layered: (i) each `stdlib` method’s doc-comment states the exact Z3 formula it must match; (ii) example-based unit tests pin the Rust side to Z3’s documented result and have caught real divergences; (iii) for the error-prone *bit-vector* families (signed vs. unsigned division/comparison; over-width shifts; `i64::MIN / -1`) we discharge the contract *empirically* with a randomized differential harness comparing `rust_impl` against the *backend-emitted* formula (§8); the cross-backend both mode is an extra whole-query check that catches pipeline regressions.

Single free input; fixed context. A load-bearing point governs *which* inputs are free in the synthesis query.

Principle 1 (Free/fixed agreement). The set of inputs treated as free by the SMT query must coincide with the set of inputs left free by the concrete oracle; everything else must be fixed to the same value on both sides.

The IMP entry point `eval_com(c: Com)` fixes the empty store internally, so the program is the single free input; were the store left free, Z3 could choose a starting state the concrete run never uses—a “witness” that does not replay. TOML’s `parse_toml` needs no wrapper.

Acceptance by Z3, cross-checked by replay.

Observation 1 (Marker-specific acceptance). A candidate is accepted for target d only if Z3 decides that the lifted semantics reach the *named* marker d , and the same candidate, re-executed through the concrete interpreter, fires the *same* d —not merely some marker.

Z3 decides the lifted reachability query; replay is an *independent* cross-check of the identical fact through the concrete reading of the spec. It re-parses and re-executes the decoded input under a wall-clock bound and checks membership of the *same* `marker_id(d)` the query asserted—so a candidate firing a *different* marker is rejected. The cross-check is load-bearing for two reasons. First, it *guards against* a violation of the lift obligation (Definition 5.1) on whatever constructs the candidate exercises: were the transpilation unfaithful on one, Z3 could validate a candidate the concrete semantics do not, and replay catches exactly that disagreement (the known `stdlib` divergence `Array::contains_key` on sentinel values thus causes only a *spurious rejection*, never a false accept). Second, quantified or depth-bounded queries can return a *candidate* model (Z3 reports `sat` with `(:reason-unknown ""))` that does not genuinely reach the marker; replay rejects

these too. Identifier and bit-vector decoding do not affect which marker fires.

Completeness: not claimed. We do *not* claim completeness in general. Even when an input reaching a marker exists, Z3 may return unknown or time out: the encoded formula can fall outside what the solver decides within its resource limits. At best we have *bounded* completeness—relative to a structural/fuel bound (§6)—and even that is contingent on the solver.

6 Bounded synthesis

Markers to queries. For a target marker with id i ($i = \text{marker_id}(d)$ for a named marker d), the query declares one constant per top-level parameter (`input_0, \dots`), applies the entry function, and asserts that the returned error value’s carried Path *equals* i (an integer equality, guarded by the enum constructor test that selects the error variant), then `(check-sat)/(get-model)`.

Bounded recursion by unrolling. A recursive interpreter is not a finite formula. A `k=N` flag selects between two routes. With `N=0` (default) recursive strongly-connected components (SCCs) are emitted as Z3 `define-funs-rec` and the solver handles recursion natively. With `N ≥ 1` every recursive SCC is unrolled into $N+1$ depth-indexed copies $f_0, \dots, f_N$: f_d calls f_{d-1} for in-SCC callees, f_0 is a terminator, and external callers route through f_N . A `sat` model under depth- N unrolling is a witness reachable in $\leq N$ recursive steps—the sense in which completeness is *bounded*, as in bounded model checking [3].

A non-error terminator makes unrolling productive. The terminator f_0 matters. When the return type has no nullary variant, the backend falls back to an *unconstrained* null sentinel, which the solver may instantiate as the very error value a marker assertion demands, so a depth- N `sat` can be satisfied *through the cutoff* rather than a genuine path. Replay rejects such candidates, so soundness is never at risk, but the budget is wasted. The remedy is a nullary *bottom* variant never produced by the concrete semantics: the cutoff can no longer discharge an `Err`-marker assertion, forcing a real marker-firing path within depth N .

Model decoding and the loop. A `sat` model is decoded to runnable source by a printer that extracts `input_0`, inlines let-shared subterms, maps mangled constructors back to AST nodes, canonicalizes solver-chosen identifiers (sound: which marker fires is spelling-independent), and decodes bit-vector literals as two’s-complement signed integers. This rendering step is visible in IMP, where Z3 returns a `Com` AST that must be printed as an IMP program. In TOML, the top-level input is already a sequence of Unicode code points, so the decoded model is the TOML document itself. The loop per target—build, solve, decode, replay, accept only if

the marker fires—then writes the accepted witnesses as a conformance suite.

7 Case studies

RuSMT is meant for authors of new executable semantics, so the case studies are selected to stress two distinct regimes rather than to maximize benchmark count. IMP is the small dynamic-semantics baseline: Z3 synthesizes an AST, RuSMT renders it back to source, and replay closes the loop. TOML tests a realistic parser with both syntactic and semantic errors: the input is already a document, solver-only search fails, and the co-solving stage produces the suite used for downstream conformance testing.

7.1 IMP: closing the loop end-to-end

IMP is the canonical small imperative “while” language of Winskel [26], chosen because its big-step semantics is familiar and recursive but small enough that native Z3 recursion should succeed if the pipeline is sound. Its grammar has arithmetic and boolean expressions, assignment, sequencing, conditionals, and while loops. We add two dynamic error branches to make conformance obligations explicit: division by zero and reading an uninitialized variable from the initially empty store. The target programs Z3 produces are correspondingly interpretable: $A := (0 / 0)$ for `division_by_zero` and $A := v0$ for `uninitialized_variable`. This study answers whether a direct-style interpreter can be lifted, solved, decoded, and replayed without bounded unrolling or proposer help.

7.2 TOML: a parser suite

TOML 1.1.0 [17] is chosen because it is large enough to defeat the solver while still having a precise public specification and several independent implementations. The executable parser has a single free input, a sequence of Unicode code points, and 151 parser functions covering documents, key-value pairs, tables, arrays, inline tables, booleans, integers, floats, strings, and datetimes. Its 182 named markers cover syntactic errors (malformed keys, unterminated strings, invalid escapes, missing array/table delimiters) and semantic violations (integer/float overflow, out-of-range date/time fields, duplicate keys, redefined tables).

This is the case study that motivates the paper’s method. Free symbolic search returns nothing for every marker, and the failure is worth stating precisely because it dictates the remedy: given a symbolic input Z3 does not report unknown after exhausting a budget, it fails to produce a verdict at all within 300 s, and on partially-symbolic variants of the same query it terminates abnormally. Given a *determined* input the same query decides in well under a second. The asymmetry is not about search space size but about macro expansion: the transpiled parsers are `define-fun` definitions that Z3 substitutes during preprocessing, before it can know which

Table 1. Main synthesis and conformance results. Stage 2 counts only candidates that Z3 decided against the original marker query with one added input equality.

Check	Setting	Result
Stdlib contract	BV, String/Seq, edge	2715 + 5317 + 216 Z3-backed checks, 0 disagreements
IMP Stage 1	$k = 0$	2/2 markers; ASTs rendered to IMP programs
TOML Stage 1	$k = 0$	0/182 markers at affordable budgets
TOML Stage 2	gpt-5.4-mini, 9 rounds	131/182 markers; 51 misses
Proposal audit	689 decided/undecided proposals	131 accepted, 551 Z3-rejected, 7 undecided; 65 duplicate proposals skipped before Z3
Conformance diff	131 tests $\times$ 4 parsers	15 accept/reject divergences

branch is taken, so a concrete input lets the simplifier fold each conditional as it expands while a symbolic one leaves both arms of every conditional alive.

The recovery route follows from that asymmetry. The model supplies a determined input from the SMT-LIB query and feedback; Z3 decides that pinned input against the original marker query. The accepted witnesses are small and readable: `a=1979-13-01` for the month-range marker, `[a]\n[a]` for the duplicate-table marker, and `a=[` for an unterminated array. These documents become the conformance suite of §8. We do not claim a verified complete TOML formalisation.

8 Evaluation and discussion

Evaluation questions. The evaluation follows the RQs from §1: whether one executable semantics can be both oracle and synthesis source (RQ1), whether Z3 acceptance plus replay avoids manual inspection (RQ2), how much the co-solving stage recovers on TOML (RQ3), whether Z3 rejects model proposals often enough to matter (RQ4), and what the generated suite finds against real implementations (RQ5).

Setup. We measured on an Apple M1 running macOS 26.5 with Z3 4.15.4, driven as a subprocess over SMT-LIB2. The IMP suite was generated by `cargo run -p rusmt-smt-derive - imp eval_com`; both markers solved in Stage 1 under native recursion ($k=0$). The TOML suite was generated by `cargo run -p rusmt-smt-derive - tom1 parse_tom1` through the resumable sweep driver with six independent workers. The TOML co-solving run used `gpt-5.4-mini`, `RUSMT_ROUNDS=9`, `RUSMT_WITNESSES=1`, and a 15 s Z3 budget for each pinned candidate. The proposer ran in an empty directory with file and shell tools disabled, so its input was the emitted SMT-LIB excerpt, the marker name, and Z3 feedback from previous rounds. We use cached transcripts for replay; exact re-generation may vary because the model is nondeterministic, so the artifact records both the generated suite and the prompt cache used for the reported run.

Contract-check results. Before the synthesis results, we confirm the lift itself: we ran the differential harness of §5 on the 64-bit bit-vector families against Z3 4.15.4: 13 signed (I64) operations and the 6 unsigned (U64) operations whose formula differs from the signed one, comparing *rust_impl* against the backend-emitted formula on directed edge cases and seeded random inputs. The harness ran 2715 operation-input pairs with zero disagreements. A companion harness covers ASCII String and Unicode Seq<U32> families with 5317 checks, again with zero disagreements, and a directed edge-class harness adds 216 more. This is not a full proof of the stdlib contract: 32-bit vectors, reals, and floats remain on documented-formula-plus-example-test evidence.

IMP: the loop closes in Stage 1. Under native recursion ($k=0$) the solver returns a model for both IMP markers and every witness replays. Both witnesses are assembled by Z3 from the datatype constructors alone, then rendered to source: $B := (0 / 0)$ reaches `division_by_zero`, and $v0 := v0$ reaches `undefined_variable` from the initially empty store. We do not run IMP through the implementation-diff harness; it is a small end-to-end demonstration of the framework, not a language ecosystem with competing production parsers.

TOML: solver-only search. Lifting the TOML parser produces SMT that Z3 accepts without frontend error, but solver-only search over the free document input produced no witness for any of the 182 markers at affordable budgets. The failure mode is sharper than ordinary timeout: with a symbolic input, macro expansion leaves both sides of deeply nested parser branches alive before simplification can fold them; with a fully determined input, the same query typically folds along one path and decides quickly. Thus Stage 2 is not replacing a successful generator. It changes the solver’s role on hard parser targets from unconstrained search to deciding a specific candidate against the original marker query.

TOML: co-solving coverage. The Stage 2 run produced 131 accepted TOML documents out of 182 markers. The 51 misses are not spread uniformly. They concentrate in three families: date/time and numeric-offset parsing (21 markers), numeric literal edge cases, especially float overflow and malformed radix forms (16 markers), and table/key redefinition or unterminated composite constructs (8 markers). The rest are isolated boolean, array-comment, and multiline-string cases. This distribution is useful for diagnosis: Stage 2 handles many local syntax errors, but remains weakest where the intended marker is easily shadowed by an earlier parse error or where the input must navigate a long chain of numeric subparsers before reaching the labelled branch. The accepted witnesses are the suite; we do not fill misses with hand-written tests, and the artifact records the full marker-level ledger.

Why the Z3 step matters. Of 689 non-duplicate candidate proposals reaching the Z3 acceptance step, only 131

Table 2. Implementation agreement on the generated TOML suite.

Implementation	Agreement	Divergences
Rust toml	130/131	1
CPython toml-lib	123/131	8
BurntSushi toml	128/131	3
Node smol-toml	128/131	3

became witnesses. Z3 rejected 551 as unsat: those inputs were valid, reached another marker, or were shadowed by an earlier error. Another 7 did not produce a useful Z3 verdict within budget. A model-only generator would have had no marker-exact way to separate these from accepted witnesses. Dynamic execution alone can say whether an input triggers some behaviour, but it cannot prove a marker unreachable on unsat, and it does not give the monotone acceptance property: every model of $Q \wedge \text{input} = c$ is a model of the original query Q .

Conformance diff. We ran the 131 generated TOML documents against four implementations: the Rust toml crate 1.1.3+spec-1.1.0, CPython toml-lib from CPython 3.14.4, BurntSushi toml v1.6.0 for Go, and smol-toml 1.7.1 for Node. Table 2 summarizes agreement with the RuSMT oracle. The suite found 15 accept/reject divergences over 131×4 implementation runs.

The clearest candidate implementation bugs are syntax and integer-boundary cases: smol-toml accepts `[[a]]`, where the oracle and the other three parsers require the closing array-table delimiter; BurntSushi accepts `a = { b = 1 }\n a.c = 2`, where the oracle treats the inline table as closed; and CPython accepts several integers outside the TOML 64-bit signed range, including `+9223372036854775808`, `-9223372036854775809`, `0x8000000000000000`, and oversized binary and octal forms. Other divergences require care before calling them parser bugs: several implementations accept extremely large positive or negative float exponents that the current RuSMT formalisation rejects, so we report those as oracle-policy/specification-triage cases rather than confirmed implementation defects. The artifact keeps the full per-input diff so maintainers can be sent minimal reports with the exact witness and oracle verdict.

Answers to the evaluation questions. RQ1 is positive for both case studies: one Rust semantics is the oracle and the source of generated tests. RQ2 is answered by marker-specific Z3 acceptance plus replay; no witness is accepted from model text alone. RQ3 is positive but not complete: co-solving covers 131/182 TOML markers after Z3-alone covers none. RQ4 is answered by the rejection count: 551 proposed inputs were rejected by Z3, so the solver is doing real work in the loop. RQ5 is the differential result: the generated suite exposes 15 accept/reject divergences across four TOML parsers.

9 Related work

Solver-aided executable semantics. Rosette [23, 24] is the closest solver-aided host: a symbolic VM compiles a language subset to constraints while concrete evaluation strips away ordinary host code. RuSMT is narrower and more conformance-oriented: the user writes a reference interpreter, marks specification obligations, and receives tests accepted only by Z3 validation plus replay. SemGuS [9] synthesizes from syntax and semantics encoded as constrained Horn clauses; RuSMT instead derives formulas from ordinary recursive functions. Synantic [11] synthesizes a semantics from a black-box interpreter, complementary to our source-to-SMT route.

Specification-derived testing. Executable-semantics frameworks such as K [18], PLT Redex [7], and Ot/Lem [14, 22] derive tools from rewrite or inference-rule specifications. RuSMT derives a more focused tool from direct-style Rust: marker-directed conformance tests. JEST [15] is closest in purpose, using a mechanized ECMAScript spec as oracle, synthesizer, and conformance tester. Our distinction is the Z3/replay acceptance gate on author-named markers and the explicit sound-but-incomplete split exposed by the TOML case study. Coverage- and grammar-driven generators such as Csmith [29] remain the application domain rather than the technique: RuSMT derives targeted witnesses from the executable specification.

Language models behind trusted checkers. We do not claim novelty for the proposer/checker pattern. Lemur [28] uses LLM-proposed verification structure, and AquaForte [12] has an LLM propose SMT models that the solver rechecks. Test-generation portfolios such as CodaMOSA [10] and Cottontail [25] similarly call a model only when conventional generation stalls. RuSMT’s modest difference is where the checked obligation lives: the same executable semantics is oracle and synthesis source, and the accepted obligation is a stable named marker rather than generic coverage. The model is useful as a generator after solver search stalls; Z3 still decides each accepted candidate against the emitted marker query.

10 Limitations and threats to validity

Scope and subset. The DSL excludes loops, mutation, and references; the price buys a clean SMT translation, mirrors textbook big-step semantics, and is mechanically enforced. RuSMT targets a specification’s *negative space*—its syntactic and semantic error branches—not happy-path fuzzing (§1). A marker covers only an error condition the author *wrote*; a violation the specification implies but the reference omits is not marked. The TOML suite is therefore marker-complete only relative to this formalisation, not to all of TOML 1.1.0. Naming TOML markers does not change the lifted query, only which targets are name-addressable.

The authoring burden. The artifact is *write-once, use-many*—one source is oracle, synthesis source, and replay oracle, amortizing over a reusable specification rather than a throwaway generator [7, 18]. The burden is lowered but not removed: a human must choose the DSL-expressible subset, place the markers, and review and own the result. The approach thus pays off when a reusable reference semantics is wanted anyway, and is poor economics for a one-off test.

Soundness validation and the solver frontier. The per-construct contract is *checked* only for the bit-vector and string/sequence families (§8), not the 32-bit variants, reals, or floats, where a divergent primitive would silently degrade faithfulness; extending the harness is the most important remaining validation work. We claim only *bounded* completeness, contingent on the solver. The dominant limitation is solver capability on deeply recursive, multi-theory queries, and we now characterise it precisely rather than reporting it as a wall: Z3 decides a *determined* input on the TOML parser quickly and returns nothing useful for a symbolic one at affordable budgets, because the transpiled parsers are macros expanded before branch conditions can fold. That diagnosis is what makes the co-solving stage principled rather than opportunistic—the model attacks search, and Z3 still decides acceptance—but it does not make the solver complete, and 51 markers remain uncovered (§8). A natural next step lets a stuck solver hand control back through Z3’s user-propagator API—callbacks at decision points the solver controls, no vendored patch—so a proposer could inject solver-*rechecked* lemmas [12, 28], or, more ambitiously, register the concrete reference semantics as a propagated theory so a candidate is replayed *inside* the search and a rejection returns a conflict clause rather than merely ending a round. Notably this needs no second encoder: Z3_solver_from_string loads the backend’s own emitted SMT-LIB into an in-memory solver to which a propagator attaches directly.

Scope of the AI integrations. The co-solving stage is measured, not only described: every reported recovery comes from live model calls, and the model is a pipeline stage rather than an optional extra. What we do *not* claim is a comparison between models, an authoring-effort measurement, or that the model’s proposals are reliable in isolation—they are not, and they do not need to be, because Z3 decides each one against the unmodified query. The proposer runs with its file and shell tools disabled in an empty working directory, so it cannot read the Rust semantics and recover an answer from the source rather than from the emitted SMT-LIB; we verified this directly. Every exchange is recorded, keyed by a content hash of the prompt, so the reported run replays without model access. Re-running the model from scratch is not guaranteed to regenerate the identical suite; the scientific result is therefore the recorded run plus the generated suite, not a claim of model determinism.

Threats to validity, by category. Read against the standard empirical-software-engineering taxonomy [19, 27]: *construct*—a marker measures an error the reference author wrote, not the specification’s true error space, and the per-construct contract is checked only for the bit-vector and string/sequence families (2715+5317+216 Z3-backed checks), so “the lift is faithful” is measured there and argued for reals, floats, and 32-bit vectors. *Internal*—acceptance is by replay under *named* markers whose id coincides on both sides by construction, so a witness is credited only when re-execution fires *that* target, excluding “some-marker” confounds and spurious k-bounded models (§8); the array-free re-encoding is controlled by byte-identical concrete output across both encodings on every test (regression-preserving, not formally proven equivalent). *External*—two case studies, one machine, one solver version, one model, no cross-machine or cross-model claim; the frontier is exhibited by TOML but argued from recursion depth $\times$ theory combination to be a property of method $\times$ solver. *Conclusion*—the TOML co-solving run is a single recorded model run, and Z3’s heuristics make latencies run-dependent, so we report the qualitative regime and content-address each query; “zero disagreement” attests no counterexample *within* the checks, not exhaustive equivalence.

11 Conclusion

RuSMT lets an author write a language’s reference semantics once, as an ordinary recursive Rust program that is at the same time the conformance oracle and the source of the tests. Z3 alone closes the loop for IMP; on the larger TOML parser, a model proposes concrete documents from the emitted SMT-LIB and Z3 decides whether each one reaches the named marker. The result is a generated TOML conformance suite covering 131/182 markers and exposing 15 accept/reject divergences across four parsers. The model is not a judge and the suite is not complete, but the accepted tests are marker-specific and replayed against the concrete oracle. That is the core promise: living executable specifications that regenerate targeted conformance suites when the spec changes.

References

- [1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udapa. 2013. Syntax-Guided Synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*. IEEE, 1–8.
- [2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6*. Technical Report. Department of Computer Science, The University of Iowa. Available at <https://smt-lib.org>.
- [3] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS, Vol. 1579)*. Springer, 193–207. doi:10.1007/3-540-49059-0_14
- [4] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 114–129. doi:10.1109/SP.2014.15
- [5] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [6] Ecma International TC39. [n. d.]. Test262: ECMAScript Conformance Test Suite. <https://github.com/tc39/test262>. Accessed 2026-06-20.
- [7] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex*. MIT Press.
- [8] Haryadi S. Gunawi, Cindy Rubio-González, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Ben Liblit. 2008. EIO: Error Handling is Occasionally Correct. In *6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 207–222.
- [9] Jinwoo Kim, Qinheping Hu, Loris D’Antoni, and Thomas W. Reps. 2021. Semantics-Guided Synthesis. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–32. doi:10.1145/3434311
- [10] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Sid-dhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering (ICSE 2023)*. IEEE. doi:10.1109/ICSE48619.2023.00085
- [11] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D’Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 362–388. doi:10.1145/3689724
- [12] Kunhang Lv, Yuhang Dong, Rui Han, Fuqi Jia, Feifei Ma, and Jian Zhang. 2026. LLM-Guided Quantified SMT Solving over Uninterpreted Functions. In *Proc. 40th AAAI Conference on Artificial Intelligence*. arXiv:2601.04675.
- [13] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [14] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: Reusable Engineering of Real-World Semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*. ACM, 175–188. doi:10.1145/2628136.2628143
- [15] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*. IEEE, 13–24. doi:10.1109/ICSE43902.2021.00015
- [16] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press.
- [17] Tom Preston-Werner et al. 2025. TOML: Tom’s Obvious, Minimal Language, Version 1.1.0. <https://toml.io/en/v1.1.0>. Language specification.
- [18] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012
- [19] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8
- [20] Len Sassaman, Meredith L. Patterson, Sergey Bratus, and Michael E. Locasto. 2013. Security Applications of Formal Language Theory. *IEEE Systems Journal* 7, 3 (2013), 489–500. doi:10.1109/JSYST.2012.2222000
- [21] Nicolas Seriot. 2016. Parsing JSON is a Minefield. http://seriot.ch/parsing_json.php. Accessed 2026-06-20.
- [22] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: Effective Tool Support for the Working Semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*. ACM, 1–12. doi:10.1145/1291151.1291155

- [23] Emina Torlak and Rastislav Bodík. 2013. Growing Solver-Aided Languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*. ACM, 135–152. doi:10.1145/2509578.2509586
- [24] Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. ACM, 530–541. doi:10.1145/2594291.2594340
- [25] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. 2025. Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation. *arXiv preprint arXiv:2504.17542* (2025). arXiv:2504.17542 To appear, IEEE Symposium on Security and Privacy (S&P) 2026.
- [26] Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press.
- [27] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. doi:10.1007/978-3-642-29044-2
- [28] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.04870
- [29] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. ACM, 283–294. doi:10.1145/1993498.1993532
- [30] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. 2014. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 249–265.